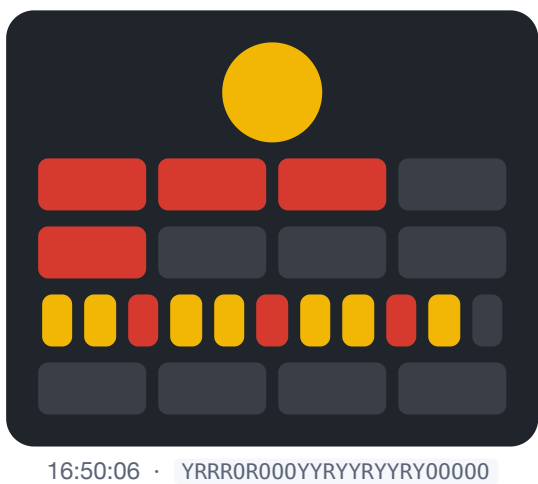


Berlin Clock

User stories

Vertical slices (React UI → Spring Boot → PostgreSQL)
that a person can test by using the running app, built test-first.



Author	Malcolm Waldron
Version	2.0 · 5 October 2026
Stack	Java 21 · Spring Boot 3.5 · JdbcClient · Flyway · PostgreSQL 17 · React 19 · TypeScript · Vite · Docker Compose
Kata source	stephane-genicot.github.io/BerlinClock.html (brief) · agilekatas.co.uk/katas/BerlinClock-Kata (scenario tables)
Method	Test-Driven Development: red → green → refactor, one vertical slice at a time

Contents

1. How to read this document	3
2. The Berlin Clock: rules and notation	3
3. Story map and delivery order	4
4. Ways of working: architecture, TDD, test strategy, Definition of Done	5
Feature A: Convert a time to the Berlin Clock	
A1 · Walking skeleton: convert a time, see the seconds lamp, find it in history	6
A2 · Clear feedback for invalid times	7
A3 · Five-hours row	8
A4 · Single-hours row	9
A5 · Five-minutes row	10
A6 · Single-minutes row	11
A7 · The entire Berlin Clock	12
Feature B: Conversion history	
B1 · Revisit a past conversion	13
B2 · Clear the history	14
Feature C: A live Berlin Clock	
C1 · Live clock ticking every second	15
C2 · Resilience when the backend or database is unavailable	16
Feature D: Hand-over to reviewers	
D1 · Explorable API documentation	17
D2 · Reviewer guide and TDD journey	18
Appendix A · Traceability: kata scenarios → stories	19
Appendix B · API and data model reference	20
Appendix C · Scope, assumptions and decisions	21

1. How to read this document

- The work is split into **four features** (A → B → C → D), each made of **vertical user stories**. Every story:
- **Is a full vertical slice**: React UI → Spring Boot REST API → domain logic → PostgreSQL, and back. Nothing is built "backend first, frontend later".
 - **Can be tested by a person**: each story ends with a **"Try it yourself"** script. Start the app with `docker compose up`, use the screen, refresh the page or restart the stack to see the data persist, and optionally look at the table with `psql`.
 - **Is also covered by automated tests** written first (TDD), including a full-stack acceptance test against a real PostgreSQL (Testcontainers).
 - **Builds only on earlier stories**: you can stop after any story and still have a working, shippable application.

Each story page contains: the user story, why it comes at this point, acceptance criteria in Gherkin (using the exact kata examples), what changes in the UI, API and database, the tests to write in TDD order, and the "Try it yourself" script. The mini clock at the top right of each story shows what the UI can display *after* that story. The **outlined** row is the one the story adds, and faded rows are not built yet.

Personas

Persona	Description
Clock reader	Anyone who wants to see a time shown the Berlin Clock way, either a time they pick or the current time, and come back to earlier conversions.
Tester / reviewer	The BNP Paribas evaluator. Downloads the code, runs it with one command, uses the app to check each story, reads the tests and the git history.
API consumer	Another system (or the React UI) calling the REST API.

2. The Berlin Clock: rules and notation

The Berlin Clock (*Mengenlehreuhr*) shows the time with illuminated coloured lamps, read from top to bottom:

Row	Lamps	Each lit lamp
Seconds lamp	1 yellow	On for even seconds, off for odd
Five hours	4 red	5 hours
Single hours	4 red	1 hour
Five minutes	11; every 3rd red (quarters), rest yellow	5 minutes
Single minutes	4 yellow	1 minute

Lamps light **from the left**. Example: 16:50:06 = 3 × 5h + 1 × 1h, 10 × 5m + 0 × 1m, even second.

TEXT NOTATION (FROM THE KATA)	
Char	Meaning
Y	Yellow lamp lit
R	Red lamp lit
0	Lamp off

The **entire clock** is a 24-character string, concatenated top to bottom:

Y

RRR0

R000

YYRYRYRYRY0

0000

→

YRRR0R000YYRYRYRYRY00000

seconds (1)

five hours (4)

single hours (4)

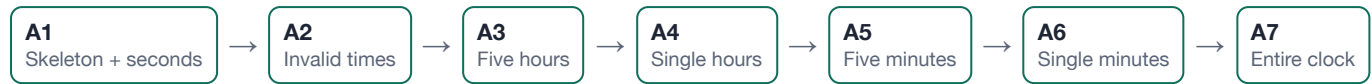
five minutes (11)

single minutes (4)

Why the rows go top to bottom. The UI grows like the real clock, one row at a time from top to bottom, in the same order as the 24-character string. The seconds lamp is also the simplest rule, which makes it the right first story for setting up the whole UI → API → database pipeline.

3. Story map and delivery order

FEATURE A: CONVERT A TIME TO THE BERLIN CLOCK



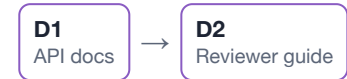
FEATURE B: CONVERSION HISTORY



FEATURE C: LIVE BERLIN CLOCK



FEATURE D: HAND-OVER

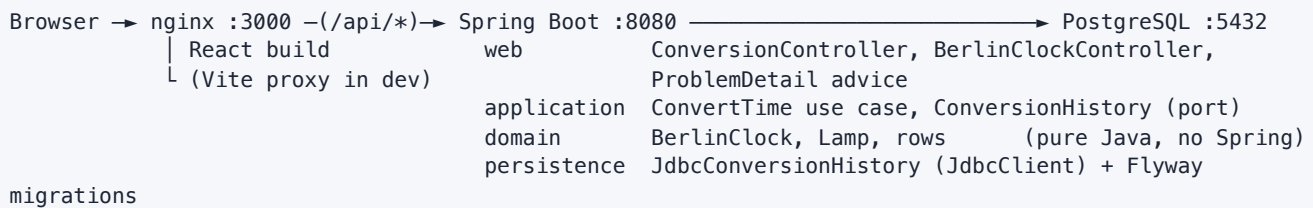


ID	Story	What a tester can do after this story	Slice: UI → API → DB
A1	Walking skeleton: seconds lamp + history	Type <code>00:00:00</code> , click Convert: the seconds lamp is yellow and the conversion appears in <i>Recent conversions</i> . Refresh or restart: it is still there.	Form + lamp + history list → POST/GET <code>/api/conversions</code> → conversion table
A2	Invalid times	Type <code>25:00:00</code> : inline error, and nothing is added to the history.	Inline error → 400 ProblemDetail → no insert
A3	Five-hours row	<code>16:35:00</code> lights <code>RRR0</code> in row 2, in the result and in the history.	New row → new field → (computed on read)
A4	Single-hours row	<code>14:35:00</code> lights <code>RRRR</code> in row 3.	same
A5	Five-minutes row	<code>12:35:00</code> shows <code>YYRYRY0000</code> with red quarter lamps.	same
A6	Single-minutes row	<code>12:34:00</code> lights <code>YYYY</code> ; the clock is visually complete.	same
A7	Entire clock	<code>16:50:06</code> shows <code>YRRR0R000YYRYRYRYRY00000</code> ; history entries show their code.	Code text → <code>clock</code> field
B1	Revisit a conversion	Click a history entry: it opens in the result panel (also via a shareable URL).	Click → GET <code>/api/conversions/{id}</code> → select by id
B2	Clear history	Click <i>Clear history</i> , confirm in the page: the list is empty, also after a refresh.	Button → DELETE <code>/api/conversions</code> → delete
C1	Live clock	Watch the current time tick; the seconds lamp blinks. History is not filled by ticks.	Live panel → GET <code>/api/berlin-clock</code> (read-only, no DB)
C2	Backend / DB down	Stop the backend or the database: clear banners and messages; start them again and the app recovers.	Banner → 503 ProblemDetail ← DB unreachable
D1	API docs	Open <code>/swagger-ui.html</code> and try every endpoint.	OpenAPI
D2	Reviewer guide	Run the app from a fresh clone using only the README; tag <code>v1.0.0</code> .	Docs

Release points. The application is shippable after every story. A6 is the first point where the UI is visually complete, A7 completes the kata's Feature 1, and D2 is the version handed to BNP Paribas.

4. Ways of working

4.1 Architecture: one vertical slice per story



- **Domain:** pure Java, framework-free. All Berlin Clock rules live here and only here.
- **Database stores facts only** (input time + when converted); the Berlin code is computed on read, so early history entries gain every row added later.
- **Ports and adapters:** the use case depends on `ConversionHistory`; tests use an in-memory fake, production the `JdbcClient` adapter.

4.2 TDD cycle and commit convention

Outside-in per story: write the failing **acceptance test** first, then work inward (domain → use case → persistence → web → UI) with one red → green → refactor cycle per example. Each step is its own commit, and pushes happen only after green/refactor so CI stays green:

```
test(red):    five hours row lights one lamp at 08:23:00
feat(green):  light one five-hours lamp per full 5 hours
refactor:     extract row-lighting helper shared by hour rows
```

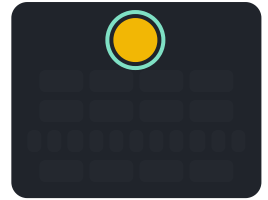
4.3 Test strategy

Layer	Tooling	What it proves
Domain unit	JUnit 5, AssertJ, @ParameterizedTest + @CsvSource	The Berlin Clock rules; every kata table row copied verbatim. 100% line/branch coverage enforced (JaCoCo).
Use case	JUnit 5 + in-memory ConversionHistory fake + fixed java.time.Clock	Save / list / find / clear behaviour without a database.
Persistence adapter	@JdbcTest + Testcontainers PostgreSQL 17 + Flyway	SQL, mapping and ordering against the real database engine.
Web slice	@WebMvcTest + MockMvc	HTTP contract: status codes, JSON shape, Location header, RFC 7807 errors.
Acceptance (full stack)	@SpringBootTest(RANDOM_PORT) + Testcontainers + RestClient	The story's acceptance criteria over real HTTP down to a real PostgreSQL, with nothing mocked.
UI component	Vitest, React Testing Library, MSW, fake timers	What the user sees: lamps, history, errors, live ticking.
Human test	docker compose up + browser (+ psql)	The story's "Try it yourself" script on the real stack.

4.4 Definition of Done (applies to every story)

- All acceptance criteria are covered by automated tests written **before** the production code (visible as red → green commits).
- `./mvnw verify` (needs Docker for Testcontainers) and `npm run lint && npm test` pass locally and in GitHub Actions.
- Domain package at 100% line and branch coverage. Schema changes only through new Flyway migrations, never by editing an applied one.
- `docker compose up --build` starts UI, API and database, and the story's "Try it yourself" script passes.
- No conversion logic in the frontend. README updated if behaviour changed. Clean Code: clear names, small functions, no dead code.

Walking skeleton: convert a time, see the seconds lamp, find it in history



after A1

KATA: SECONDS LAMP

SETS UP UI → API → DB PIPELINE

As a clock reader, **I want** to convert a time, see the Berlin Clock's seconds lamp and find my conversion in a history, **so that** I can tell whether the second is even or odd and come back to it later.

Depends on: nothing. **Why first:** the seconds lamp is the simplest rule, so this story can concentrate on building the whole pipeline (React, Spring Boot, PostgreSQL, Flyway, Docker Compose, CI) through one real feature.

ACCEPTANCE CRITERIA

Scenario Outline: The seconds lamp blinks every second

When I convert <time>

Then the seconds lamp is <lamp>

Examples:

time	lamp	
00:00:00	Y	← kata 12:00:02 Y ← extra: any even second
23:59:59	0	← kata 12:00:01 0 ← extra: any odd second

Scenario: A conversion is saved and listed, newest first

Given I converted 00:00:00 and then 23:59:59

When I look at "Recent conversions"

Then I see 23:59:59 above 00:00:00, each with its seconds lamp and when it was converted

And they are still listed after I refresh the page or restart the whole stack

Scenario: Only the 10 most recent conversions are listed

Given I made 11 conversions

Then "Recent conversions" shows the latest 10, and the first conversion is not listed

UI · API · DB

```
POST /api/conversions {"time":"00:00:00"}
201 Created Location: /api/conversions/1
{ "id": 1, "time": "00:00:00",
  "convertedAt": "2026-10-05T14:03:12Z",
  "seconds": "Y" }
```

```
GET /api/conversions?limit=10 → 200 [ ...newest
first... ]
```

```
V1__create_conversion.sql
conversion(id BIGINT IDENTITY PK,
           time TIME NOT NULL,
           converted_at TIMESTAMPTZ NOT NULL)
```

UI: HH:mm:ss text field + **Convert** (Enter submits), result panel with the round seconds lamp, *Recent conversions* list (time, mini lamp, converted-at in browser local time).

TESTS, IN TDD ORDER

- Acceptance:** POST 00:00:00 → 201, seconds="Y"; GET lists it.
- Domain:** Lamp enum; `BerlinClock.of(LocalTime).seconds()`, one cycle per example.
- Use case:** convert saves time + `Clock.now()`; list returns newest first, max 10 (fake port).
- Persistence:** `JdbcConversionHistory` save / latest(n) on Testcontainers Postgres.
- Web slice:** 201 + Location; list JSON.
- UI:** lamp lit/unlit; history renders and refreshes after Convert.

ENABLERS

Monorepo backend/ + frontend/ ; docker-compose.yml with db (postgres:17, health check), backend, frontend (nginx proxies /api); GitHub Actions; JaCoCo gate; README quick start.

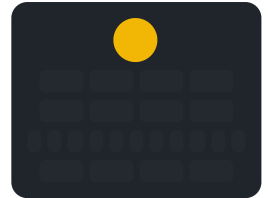
TRY IT YOURSELF

```
$ docker compose up --build
Type 00:00:00 → Convert conversions"
Type 23:59:59 → Convert
$ docker compose restart
$ docker compose exec db psql -U berlin -c "select * from conversion" → 2 rows
```

→ open http://localhost:3000
→ the seconds lamp is yellow; the entry appears under "Recent conversions"
→ the lamp is off; 23:59:59 is now at the top of the list
→ refresh the page: both entries are still there

Clear feedback for invalid times

ROBUSTNESS



after A2 (no new row)

As a clock reader, **I want** a clear message when the time I type is not valid, **so that** I know how to correct it, and my history is not filled with junk.

Depends on: A1. **Why now:** every later story uses the same input, so the input contract is fixed before any more rules are added. **Decision:** 24:00:00 is rejected: the kata examples stop at 23:59:59 and `java.time.LocalDateTime` has no 24:00.

ACCEPTANCE CRITERIA

Scenario Outline: Invalid times are rejected and not saved

When I convert "<input>"

Then the request is rejected as a bad request (HTTP 400)

And I am told: Invalid time '<input>': expected HH:mm:ss between 00:00:00 and 23:59:59

And "Recent conversions" is unchanged

Examples:

25:00:00	hour out of range	1:2:3	not zero-padded
24:00:00	24:00 not supported	12-00-00	wrong separator
12:60:00	minute out of range	12:00	seconds missing
12:00:60	second out of range	noon	not a time

Scenario: The time is missing

When I convert with an empty field (or a request without "time")

Then the request is rejected (HTTP 400) and I am told: A time is required (HH:mm:ss)

Scenario: The UI shows the error and no stale result

Given the result for 00:00:00 is displayed

When I convert "25:00:00"

Then the error is shown next to the input (announced via `role="alert"`) and the old result is cleared

UI · API · DB

```
POST /api/conversions {"time":"25:00:00"}
400 Bad Request application/problem+json
{ "title": "Invalid time", "status": 400,
  "detail": "Invalid time '25:00:00': expected
            HH:mm:ss between 00:00:00 and
23:59:59",
  "instance": "/api/conversions" }
DB: no row inserted.
```

TESTS, IN TDD ORDER

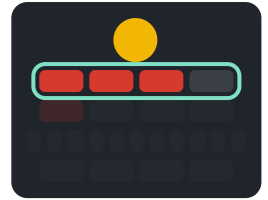
1. **Acceptance:** 25:00:00 → 400 problem+json; history count unchanged.
2. **Domain:** `DigitalTime.parse(String) : strict HH:mm:ss (ResolverStyle.STRICT)`, one test per invalid example.
3. **Use case:** invalid input never reaches the port (fake stays empty).
4. **Web slice:** `@RestControllerAdvice` → `ProblemDetail` for invalid / missing time.
5. **UI:** `detail` rendered in an alert; old result removed.

TRY IT YOURSELF

Type 12-00-00 → Convert → "Invalid time '12-00-00': expected HH:mm:ss ..."; the history list does not change
 Clear the field → Convert → "A time is required (HH:mm:ss)"

Five-hours row

KATA: FIVE HOURS ROW



after A3 · 16:00:00

As a clock reader, **I want** to see the five-hours row for the time I convert, **so that** I can read the hour in blocks of five.

Depends on: A2. **Rule:** 4 red lamps; one lit (R) per full five hours ($\text{hours} \div 5$), lit from the left.

ACCEPTANCE CRITERIA

Scenario Outline: Implement the Five Hours Row

When I convert <time>

Then the five hours row is <row>

Examples:

(all from the kata)

time	row
00:00:00	0000
23:59:59	RRRR
02:04:00	0000
08:23:00	R000
16:35:00	RRR0

Scenario: Earlier conversions gain the new row

Given 23:59:59 was converted before this story was released

Then its entry in "Recent conversions" now shows the five hours row RRRR

UI · API · DB

```
POST /api/conversions {"time":"16:35:00"}
201 { "id": 7, "time": "16:35:00",
      "convertedAt": "...",
      "seconds": "Y",
      "fiveHours": "RRR0" }      ← new field
```

UI: row of four rectangular lamps under the seconds lamp, in the result *and* in each history entry. A generic `<LampRow>` is introduced here and reused for every later row. **DB:** no change (the code is computed on read).

TRY IT YOURSELF

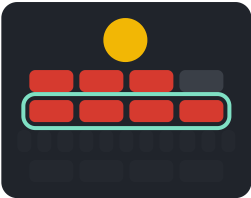
Type 16:35:00 → Convert → three red lamps and one dark lamp in row 2
Look at the older 23:59:59 entry in the history → its five hours row now shows four red lamps

TESTS, IN TDD ORDER

1. **Acceptance:** POST 16:35:00 → `fiveHours = "RRR0"` ; GET list contains it.
2. **Domain:** one red→green cycle per kata row; expected refactor: a "light *n* of *m* lamps" helper.
3. **Web slice:** JSON contains `fiveHours` .
4. **UI:** `LampRow` renders 4 lamps with the right colour per character.

Single-hours row

KATA: SINGLE HOURS ROW



after A4 · 19:00:00

As a clock reader, I want to see the single-hours row, so that I can read the exact hour.

Depends on: A3. Rule: 4 red lamps; one lit (R) per hour left over after the five-hour blocks (hours mod 5).

ACCEPTANCE CRITERIA

Scenario Outline: Implement the Single Hours Row

When I convert <time>
Then the single hours row is <row>

Examples: (all from the kata)

time	row
00:00:00	0000
23:59:59	RRR0
02:04:00	RR00
08:23:00	RRR0
14:35:00	RRRR

UI · API · DB

```
201 { "id": 8, "time": "14:35:00",
      "convertedAt": "...",
      "seconds": "Y", "fiveHours": "RR00",
      "singleHours": "RRRR" } ← new field
```

UI: third row added with the existing <LampRow> in the result and the history. **DB:** no change.

TESTS, IN TDD ORDER

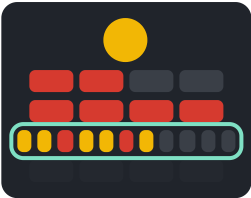
- 1. **Acceptance:** POST 14:35:00 → singleHours = "RRRR".
- 2. **Domain:** one cycle per kata row; reuse the helper from A3.
- 3. **Web slice:** JSON contains singleHours.
- 4. **UI:** third row rendered from singleHours.

TRY IT YOURSELF

Type 14:35:00 → Convert → rows 2 and 3 read RR00 / RRRR (= 2 × 5 + 4 = 14 h)
Type 02:04:00 → Convert → row 2 all dark, row 3 RR00

Five-minutes row

KATA: FIVE MINUTES ROW



after A5 · 14:35:00

As a clock reader, I want to see the five-minutes row with its red quarter markers, so that I can read minutes in blocks of five and spot quarter, half and three-quarter hours at a glance.

Depends on: A4. **Rule:** 11 lamps; one lit per full five minutes (minutes ÷ 5). Lit lamps in positions 3, 6 and 9 are red (R); all other lit lamps are yellow (Y).

ACCEPTANCE CRITERIA

Scenario Outline: Implement the Five Minutes Row

When I convert <time>

Then the five minutes row is <row>

Examples:

time	row	
00:00:00	00000000000	← kata
23:59:59	YYRYYYRYYY	← kata
12:04:00	00000000000	← kata
12:23:00	YYRY0000000	← kata
12:35:00	YYRYRY00000	← kata
12:15:00	YYR00000000	← extra: first quarter lamp is red

UI · API · DB

```
201 { ..., "time": "12:35:00",
        "fiveHours": "RR00", "singleHours":
        "RR00",
        "fiveMinutes": "YYRYRY0000" } ← new field
```

UI: fourth row of 11 narrower lamps (yellow / red / dark). Lit vs off also differs in brightness, not only hue. **DB:** no change.

TESTS, IN TDD ORDER

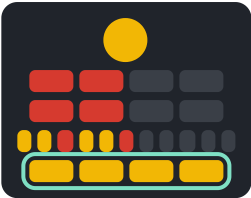
- Acceptance:** POST 12:35:00 → fiveMinutes = "YYRYRY0000" .
- Domain:** cycles: all-off, then lamp counts, then the quarter red rule (the extra 12:15 case forces it explicitly).
- Web slice:** JSON contains fiveMinutes .
- UI:** 11 lamps; positions 3/6/9 red when lit.

TRY IT YOURSELF

Type 12:35:00 → Convert → row 4 shows Y Y R Y Y R Y then four dark lamps
Type 12:15:00 → Convert → Y Y R: the quarter-hour marker is red

Single-minutes row

KATA: SINGLE MINUTES ROW



after A6 · 12:34:00

As a clock reader, I want to see the single-minutes row, so that I can read the time to the exact minute on a complete clock.

Depends on: A5. **Rule:** 4 yellow lamps; one lit (Y) per minute left over after the five-minute blocks (minutes mod 5). **Milestone:** the UI is now visually complete.

ACCEPTANCE CRITERIA

Scenario Outline: Implement the Single Minutes Row

When I convert <time>

Then the single minutes row is <row>

Examples: (all from the kata)

time	row
00:00:00	0000
23:59:59	YYYY
12:32:00	YY00
12:34:00	YYYY
12:35:00	0000

UI · API · DB

```
201 { ..., "time": "12:34:00",
      "fiveHours": "RR00", "singleHours":
"RR00",
      "fiveMinutes": "YYYYR00000",
      "singleMinutes": "YYYY" }      ← new field
```

UI: bottom row of 4 yellow lamps; all five parts of the clock rendered in the result and history. **DB:** no change.

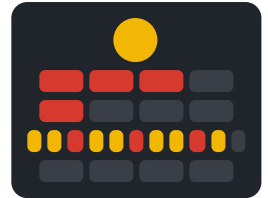
TESTS, IN TDD ORDER

- 1. **Acceptance:** POST 12:34:00 → singleMinutes = "YYYY" .
- 2. **Domain:** one cycle per kata row; single-hours and single-minutes rules now share one abstraction (refactor).
- 3. **Web slice:** JSON contains singleMinutes .
- 4. **UI:** bottom row rendered; full layout matches the reference picture.

TRY IT YOURSELF

Type 12:34:00 → Convert → the complete clock reads 12:34 (2 × 5h + 2h, 6 × 5m + 4m)
Scroll the history → every earlier entry now shows a complete clock

The entire Berlin Clock

KATA: INTEGRATE THE ENTIRE CLOCK

after A7 · 16:50:06

As a clock reader or API consumer, **I want** the whole Berlin Clock as a single 24-character code, **so that** I can store, compare or share it, exactly as the kata defines it.

Depends on: A6. **Rule:** seconds + five hours + single hours + five minutes + single minutes (1 + 4 + 4 + 11 + 4 = 24 characters).
Milestone: the kata's Feature 1 is complete.

ACCEPTANCE CRITERIA

Scenario Outline: Integrate the Entire Berlin Clock

When I convert <time>

Then the clock is <clock>

Examples: (all from the kata)

time	clock
00:00:00	Y0000000000000000000000
23:59:59	ORRRRRR0YYRYYRYYRYYRYY
16:50:06	YRRR0R000YYRYYRYYR00000
11:37:01	ORR00R000YYRYYR0000YY00

Scenario: The clock is readable without seeing colours

Given I converted 16:50:06

Then the clock is exposed as an image labelled "Berlin Clock showing 16:50:06"

And the 24-character code is shown as text below the clock and in its history entry, with a copy button

UI · API · DB

```
201 {
  "id": 12, "time": "16:50:06",
  "convertedAt": "2026-10-05T14:03:12Z",
  "clock":      "YRRR0R000YYRYYRYYR00000",
  "seconds":    "Y",
  "fiveHours": "RRR0", "singleHours":
  "R000",
  "fiveMinutes": "YYRYYRYYR0",
  "singleMinutes": "0000" }
```

DB: no change.

TESTS, IN TDD ORDER

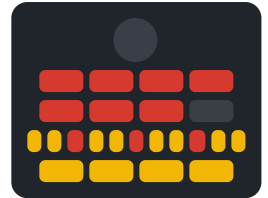
1. **Acceptance:** all four kata examples via POST and via the GET list.
2. **Domain:** `BerlinClock.code()` per kata row; property check: length is 24 for every second of the day (86,400 cases, still fast).
3. **Web slice:** JSON contains `clock`.
4. **UI:** code text + copy button; `role="img"` + `aria-label`.

TRY IT YOURSELF

Type 16:50:06 → Convert → clock + "YRRR0R000YYRYYRYYR00000" underneath; click copy and paste it somewhere
Turn on VoiceOver (Cmd+F5) → focus the clock → "Berlin Clock showing 16:50:06"

Revisit a past conversion

HISTORY



reopened · 23:59:59

As a clock reader, **I want** to open a conversion from my history (or from a link someone shared), **so that** I can look at it in full again without retyping it.

Depends on: A7.

ACCEPTANCE CRITERIA

Scenario: Open a conversion from the history

Given "Recent conversions" lists 23:59:59

When I click that entry

Then the result panel shows the full clock ORRRRRR0YYRYRYRYYYYYY for 23:59:59

And the address bar shows /conversions/<id>

And no new conversion is added to the history

Scenario: Open a shared link

Given conversion <id> exists

When I open <http://localhost:3000/conversions/<id>> in a new tab

Then the result panel shows that conversion

Scenario: Open a conversion that does not exist

When I open /conversions/999999

Then I am told "Conversion 999999 not found" (HTTP 404 from the API)

UI · API · DB

```
GET /api/conversions/12 → 200 { "id": 12,
  "time": "16:50:06", ... }
GET /api/conversions/999999
→ 404 application/problem+json
{ "title": "Conversion not found",
  "detail": "Conversion 999999 not found" }
```

```
SELECT id, time, converted_at FROM conversion
WHERE id = ?
```

UI: history entries are links; minimal client-side route
/conversions/:id (nginx falls back to index.html).

TESTS, IN TDD ORDER

1. **Acceptance:** POST then GET by the Location URL → same body; unknown id → 404.
2. **Use case:** find by id → present / empty (fake port).
3. **Persistence:** findById on Testcontainers Postgres.
4. **Web slice:** 200 / 404 ProblemDetail.
5. **UI:** clicking an entry shows it and updates the URL; deep link loads it; 404 message.

TRY IT YOURSELF

Click an older entry in "Recent conversions" → it is shown in full; the URL changes to /conversions/<id>
Copy the URL into a new tab → the same conversion opens. Change the id to 999999 → "Conversion 999999 not found"

Clear the history

HISTORY

As a clock reader or tester, **I want** to clear the conversion history, **so that** I can start again from a clean slate.

Depends on: B1.

ACCEPTANCE CRITERIA

Scenario: Clearing asks for confirmation inside the page
Given "Recent conversions" lists at least one conversion
When I click "Clear history"
Then I am asked "Delete all conversions? This cannot be undone." with Delete and Cancel buttons

Scenario: Confirming clears the history permanently
When I click Delete
Then "Recent conversions" says "No conversions yet"
And it is still empty after I refresh the page or restart the stack

Scenario: Cancelling keeps everything
When I click Cancel
Then the history is unchanged

Scenario: Nothing to clear
Given the history is empty
Then the "Clear history" button is disabled

UI · API · DB

```
DELETE /api/conversions → 204 No Content
DELETE FROM conversion
```

UI: *Clear history* button with an inline confirmation (no browser `confirm()` dialog); empty state "No conversions yet". A previously opened `/conversions/<id>` now shows "not found".

TESTS, IN TDD ORDER

1. **Acceptance:** POST ×2, DELETE → 204, GET list → [].
2. **Use case:** clear empties the port.
3. **Persistence:** `deleteAll` on Testcontainers Postgres.
4. **Web slice:** 204, idempotent (DELETE on empty → 204).
5. **UI:** confirm / cancel / disabled-when-empty / empty state.

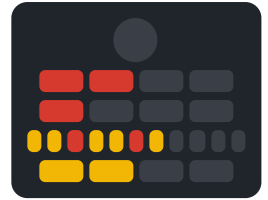
TRY IT YOURSELF

Click "Clear history" → Cancel → nothing changes.
\$ docker compose restart → refresh → still empty.
count(*) from conversion" → 0

Click again → Delete → "No conversions yet"
\$ docker compose exec db psql -U berlin -c "select

Live clock ticking every second

LIVE



live · 11:37:01

As a clock reader, **I want** the page to show the current time as a live Berlin Clock, **so that** I can use it as an actual clock and watch the seconds lamp blink.

Depends on: A7. **Design:** the browser sends its own local time each second to a **read-only** endpoint. Conversion logic stays in the backend, there is no server time-zone confusion inside Docker, and ticks are **not** saved (86,400 rows a day per tab would bury the real history).

ACCEPTANCE CRITERIA

Scenario: The live clock shows the current local time

Given my browser's local time is 16:50:06

When I open the page

Then the live clock shows YRRR0R000YYRYYRYRY00000 and the digital time 16:50:06

Scenario: The live clock ticks

Given the live clock shows 16:50:06

When one second passes

Then it shows 16:50:07 and the seconds lamp is off

Scenario: Ticks are not saved

Given the live clock has been running for a minute

Then "Recent conversions" has no new entries

Scenario: Late responses never move the clock backwards

Given the response for 16:50:07 arrives after the response for 16:50:08

Then the live clock keeps showing 16:50:08

UI · API · DB

```
GET /api/berlin-clock?time=16:50:06
200 { "time": "16:50:06", "clock": "YRRR0...",
      "seconds": "Y", "fiveHours": "RRR0", ... }
(400 ProblemDetail for invalid time, as in A2)
DB: not touched.
```

UI: a **Live** panel above the converter; `useLiveBerlinClock` hook (interval aligned to the second, cleared on unmount, ignores out-of-order responses).

TESTS, IN TDD ORDER

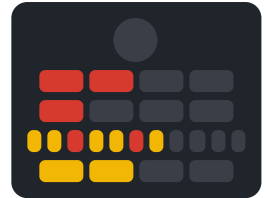
1. **Acceptance:** GET 16:50:06 → kata code; conversion count unchanged.
2. **Web slice:** 200 / 400 for the read-only endpoint (reuses domain + parsing).
3. **UI:** fake timers + `vi.setSystemTime` + MSW: first render, +1 s tick, out-of-order responses, interval cleared on unmount.

TRY IT YOURSELF

Open `http://localhost:3000` → the live clock ticks; the seconds lamp blinks once per second
Wait a minute → "Recent conversions" is unchanged. DevTools → Network → one small GET per second

Resilience when the backend or database is unavailable

LIVE



frozen at last known state

As a clock reader, **I want** to be told clearly when something is down, **so that** I never mistake a frozen clock or an unsaved conversion for a working one.

Depends on: C1.

ACCEPTANCE CRITERIA

Scenario: The backend becomes unreachable

- When** the backend stops responding (network error or HTTP 5xx)
- Then** a banner says "Backend unavailable. Retrying..."
- And** the live clock keeps its last known state, visibly dimmed, and keeps retrying every second

Scenario: The database is unavailable but the backend is up

- Given** the database is stopped
- Then** the live clock keeps ticking (it does not need the database)
- When** I convert 12:00:00
- Then** I am told "History is temporarily unavailable. Your conversion was not saved." (HTTP 503)
- And** "Recent conversions" shows "History unavailable" instead of an empty list

Scenario: Recovery

- When** the backend and database are available again
- Then** the banners disappear, the live clock is current and conversions are saved again

UI · API · DB

```
POST /api/conversions (DB down)
→ 503 application/problem+json
{ "title": "History unavailable",
  "detail": "History is temporarily
            unavailable.
            Your conversion was not saved." }
Unexpected errors → 500 ProblemDetail, no stack
trace.
```

Hikari connection timeout kept short (~2 s) so the user gets fast feedback. The backend starts even if the DB is late (compose health check + Flyway retry).

TRY IT YOURSELF

```
$ docker compose stop db          → live clock keeps ticking; Convert → "History is temporarily
unavailable..."
$ docker compose stop backend    → "Backend unavailable. Retrying..." and a dimmed clock
$ docker compose start db backend → banners clear; Convert saves again
```

TESTS, IN TDD ORDER

1. **Web slice:** `DataAccessException` → 503 `ProblemDetail`; other exceptions → 500 without internals.
2. **Acceptance:** stop the Testcontainers DB → POST returns 503; GET `/api/berlin-clock` still 200.
3. **UI:** MSW network error → banner + dimmed clock; recovery clears it; 503 on convert / list → messages above.

Explorable API documentation

HAND-OVER

As a reviewer or API consumer, **I want** interactive, accurate API documentation, **so that** I can understand and try every endpoint without reading the source code.

Depends on: C2 (the API contract is final).

ACCEPTANCE CRITERIA

Scenario: Interactive documentation is available

When I open `http://localhost:8080/swagger-ui.html`

Then I see `POST /api/conversions`, `GET /api/conversions`, `GET /api/conversions/{id}`, `DELETE /api/conversions` and `GET /api/berlin-clock`, each with examples and error schemas

And I can execute every request from the page

Scenario: The OpenAPI document is machine-readable

When I request `/v3/api-docs`

Then I receive a valid OpenAPI 3 document describing all endpoints

Scenario: The README contains copy-paste examples

Then the README shows curl examples for every endpoint, including an invalid time, with their responses

IMPLEMENTATION NOTES

- `springdoc-openapi-starter-webmvc-ui`; annotations on controllers and DTOs only (domain stays annotation-free).
- `time` documented with the pattern `^\d{2}:\d{2}:\d{2}$` and the 16:50:06 example.

TESTS

1. **Acceptance:** `/v3/api-docs` returns 200 and lists all five operations.
2. **Acceptance:** `/swagger-ui.html` is served.

TRY IT YOURSELF

Open `http://localhost:8080/swagger-ui.html` → `POST /api/conversions` → Try it out → `{"time":"16:50:06"}` → Execute → 201
Then `GET /api/conversions` → the new entry is first; it also appears in the UI history

Reviewer guide and TDD journey

HAND-OVER

As a BNP Paribas reviewer, **I want** to download, run and assess the solution in minutes, **so that** I can focus on the quality of the code and the TDD process.

Depends on: D1. **Milestone:** release v1.0.0 , the version handed over.

ACCEPTANCE CRITERIA

Scenario: Run with Docker from a fresh clone

Given a machine with only Git and Docker installed

When I follow the README quick start (git clone → docker compose up --build)

Then UI, API and database start, and the app is available at http://localhost:3000

Scenario: Run natively for development

Given JDK 21, Node.js 22+ and Docker are installed

When I run "docker compose up -d db", "./mvnw spring-boot:run" in backend/ and "npm ci && npm run dev" in frontend/

Then the app is available at http://localhost:5173

Scenario: Run all tests with one command per app

When I run "./mvnw verify" (Docker running, for Testcontainers) and "npm test"

Then all tests pass and coverage reports are produced

Scenario: Follow the TDD journey

When I open docs/tdd-journey.md

Then every story and kata scenario links to its red, green and refactor commits

README SECTIONS

1. What this is (with screenshot)
2. Quick start (Docker)
3. Run natively
4. Run the tests + coverage reports
5. Using the app: a 5-minute tour covering every story
6. API (curl + Swagger)
7. Project structure and architecture
8. Design decisions and scope
9. Troubleshooting (ports 3000/5432/8080 in use, JDK version, Docker not running, Apple Silicon)

DONE WHEN

- ☐ Fresh-clone check run on a clean machine / container
- ☐ CI badge green on `main`
- ☐ docs/tdd-journey.md complete
- ☐ This stories PDF included in docs/
- ☐ Tag v1.0.0 created

Appendix A · Traceability: kata scenarios → stories

Every example from the kata's Feature 1 is an automated test, written verbatim, in the story listed below.

Kata scenario	Time	Expected	Story	Test class
Seconds lamp	00:00:00	Y	A1	SecondsLampTest
	23:59:59	0		
Five hours row	00:00:00	0000	A3	FiveHoursRowTest
	23:59:59	RRRR		
	02:04:00	0000		
	08:23:00	R000		
	16:35:00	RRR0		
Single hours row	00:00:00	0000	A4	SingleHoursRowTest
	23:59:59	RRR0		
	02:04:00	RR00		
	08:23:00	RRR0		
	14:35:00	RRRR		
Five minutes row	00:00:00	0000000000	A5	FiveMinutesRowTest
	23:59:59	YYRYRYRYRY		
	12:04:00	0000000000		
	12:23:00	YYRY000000		
	12:35:00	YYRYRY0000		
Single minutes row	00:00:00	0000	A6	SingleMinutesRowTest
	23:59:59	YYYY		
	12:32:00	YY00		
	12:34:00	YYYY		
	12:35:00	0000		
Entire Berlin Clock	00:00:00	Y000000000000000000000000	A7	BerlinClockTest , ConversionApiAcceptanceTest
	23:59:59	0RRRRRRR0YYRYRYRYRYYYYYY		
	16:50:06	YRRR0R000YYRYRYRYRY00000		
	11:37:01	0RR00R000YYRYRY0000YY00		

Appendix B · API and data model reference

Endpoints (final state)

Endpoint	Success	Errors	Story
POST /api/conversions body {"time":"HH:mm:ss"}	201 + Location + conversion	400 invalid / missing time · 503 DB down	A1, A2
GET /api/conversions? limit=10	200, array, newest first (limit 1–50, default 10)	400 bad limit · 503 DB down	A1
GET /api/conversions/{id}	200 conversion	404 not found · 503 DB down	B1
DELETE /api/conversions	204 (idempotent)	503 DB down	B2
GET /api/berlin-clock? time=HH:mm:ss	200 clock (not saved)	400 invalid / missing time	C1

All errors use RFC 7807 / 9457 application/problem+json with title, status, detail and instance.

Conversion resource (after A7)

```
{
  "id":          12,
  "time":        "16:50:06",
  "convertedAt": "2026-10-05T14:03:12Z",      ← UTC, ISO-8601; UI shows browser local time
  "clock":       "YRRROR000YYRYYRYRY00000",
  "seconds":     "Y",
  "fiveHours":   "RRRO",
  "singleHours": "R000",
  "fiveMinutes": "YYRYRYRYRY0",
  "singleMinutes": "0000"
}
```

Database schema

```
-- V1__create_conversion.sql  (A1, the only migration planned)
CREATE TABLE conversion (
  id          BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  time        TIME(0)     NOT NULL,
  converted_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX conversion_converted_at_idx ON conversion (converted_at DESC, id DESC);
```

Only facts are stored. Every Berlin Clock field is computed by the domain on read, so there is one source of truth and no stale codes. Ordering is converted_at DESC, id DESC, which stays deterministic when two conversions share a timestamp.

Appendix C · Scope, assumptions and decisions

In scope

- The Berlin Clock rules from the challenge brief (stephane-genicot.github.io/BerlinClock.html), driven by the scenario tables of agilekatas.co.uk Feature 1, "Converting Digital Time to Berlin Time".
- A React UI to convert a time, keep a persisted history of conversions and show a live clock, backed by a Spring Boot REST API and PostgreSQL.

Out of scope (deliberately)

- **Agilekatas Feature 2, "Converting Berlin Time to Digital Time"**. It is not part of the challenge brief. It is also under-specified: one seconds lamp only shows even or odd, so outputs like `...YYYYYY → 23:59:59` cannot be derived from the lamps.
- Users and authentication (the history is shared by everyone using the instance), editing conversions, pagination beyond the latest 50, cloud deployment, internationalisation.
- Browser-automated end-to-end tests (Playwright/Cypress). Each story is verified by full-stack acceptance tests against real PostgreSQL plus a human "Try it yourself" script.

Decisions

Decision	Rationale
Every story is a slice: UI → Spring → PostgreSQL	A tester can verify each story by using the app, including persistence across refresh and restart.
Database holds conversion history	The Berlin Clock calculation is pure. Storing what users converted is the natural, useful thing to persist.
Store input time + timestamp; compute the clock on read	Store facts, derive the rest: one source of truth, no stale data, older entries gain rows as stories land.
PostgreSQL 17 + Flyway + Testcontainers	Production-grade engine, versioned schema, tests against the same engine as runtime (no H2 dialect drift).
Spring <code>JdbcClient</code> behind a <code>ConversionHistory</code> port	One small table: explicit SQL, no ORM magic. The port keeps the use case testable with an in-memory fake.
<code>POST</code> saves, <code>GET</code> reads; separate read-only <code>GET /api/berlin-clock</code>	Correct HTTP semantics (no side effects on GET). The live clock does not write to the database.
Live clock ticks not saved	86,400 rows a day per tab would bury the real history.
Strict <code>HH:mm:ss</code> , <code>00:00:00–23:59:59</code>	Matches every kata example; <code>24:00:00</code> rejected.
RFC 7807 ProblemDetail; 503 when DB is down	Standard errors; the user can tell "your input is wrong" from "our side is down".
Same-origin proxy (Vite in dev, nginx in Docker)	No CORS configuration; the frontend always calls relative <code>/api</code> .
Red / green / refactor commits; 100% gate on the domain	The TDD process can be checked from <code>git log</code> ; the gate is where 100% is meaningful.